I2AI 25/26-F AI Agent Project: Computational Analysis of Alpha-Beta Pruning Heuristics in Connect 4

Asrınalp Şahin[1], Oğuzhan Sarıgöl[2]
[1]Department of Computer Engineering, Eskisehir Osmangazi University, Eskişehir, Türkiye
[2]Department of Computer Engineering, Eskisehir Osmangazi University, Eskişehir, Türkiye
[1]asrinalpsahin@gmail.com, [2]oguzhansarigol1@gmail.com

***Abstract* - This study presents an artificial intelligence agent designed for the Connect 4 game. The agent is based on the Minimax algorithm and alpha-beta pruning approaches. Various search optimizations, including move sequencing, killer moves, dynamic depth, transposition tables, bitboarding, and threat detection, are used to reduce the Minimax algorithm's thinking time and enable more accurate decisions by using a more efficient heuristic formulation. In comparisons conducted across six different matches, win rates and average thinking times are compared. The results clearly demonstrate that search supported by well structured heuristics consistently outperforms other methods with high iteration numbers. Moreover, it has been observed that aggressive caching approaches like bitboarding can negatively impact potential wins, but with proper integration, it has been determined that thinking time can be significantly reduced. The findings highlight the importance of adaptive heuristic components in agents using Minimax.**

Keywords— Alpha-Beta pruning, Minimax algorithm, Connect 4, heuristic evaluation, move ordering, AI agents

I. Introduction

Minimax programs, heuristic algorithms, and Artifical Intelligence agents have long been researched, and interest in these areas continues to grow each year. Recent advancements in search and machine development techniques in artificial intelligence have paved the way for the emergence of systems capable of superhuman performance in many tasks. For example, Deep Blue, designed by IBM, beat world chess champion Garry Kasparov in 1997. Moreover, as minimax was constantly being studied by humans through self-play, Google's DeepMind Company designed the state-of-the-art intelligent player Alphazero. [1]

Two-player zero-sum board games provide researchers with a broad range of workspaces by providing reliable platforms where strategic thinking, situational assessment, and predictive mechanisms can be tested. Connect 4 is a frequently preferred testbed in this context. This study examines the development process of an artificial intelligence agent designed for Connect 4 and enhanced with various optimizations. The core of the agent is based on Alpha-Beta pruning with Minimax algorithm and is supported by modern techniques such as move sequencing, killer moves, transposition table, bitboard integration and threat detection to increase the speed and accuracy of the decision-making process.

In this study, the artificial intelligence agent developed for the Connect 4 game was tested using classical alpha-beta pruning and optimized versions of different algorithms. The aim was to compare different approaches and determine which methods produced more effective results. First, the algorithms and optimization techniques prominent in the relevant literature were examined. Then, the main structure of the developed system and the different approaches used were discussed. In the following sections, experimental results are presented with measurement metrics and graphs. Finally, the advantages of the proposed structure are discussed.

II. Related Work

Research on improving the performance of game-playing AI agents has focused primarily on medium-complexity zero-sum games like Connect 4. Various studies have evaluated different versions of algorithms such as Minimax, Monte Carlo Tree Search (MCTS), and Q-Learning in the literature. Taylor and Stella's study provides an evaluation framework that compares both the basic and advanced versions of Minimax, Q-Learning, and MCTS on Connect 4. In this context, they report that MCTS variants generally have the highest win rate, Minimax exhibits strong stability when configured correctly, and Q-Learning, while superior in decision speed, lags behind in playing power.[2] The same study also shows that improvements such as Alpha-Beta pruning, move sequencing, and killer moves significantly improve Minimax performance; furthermore, UCT-based MCTS provides more stable results in situations requiring deep search.

Studies focusing on Minimax and MCTS comparisons on Connect 4 also hold a significant place in the literature. Sheoran and colleagues developed a version of the Minimax algorithm that can perform deeper searches by optimizing it with dynamic programming. Their experiments reported that while the classic MCTS agent excels at low time constraints, the improved Minimax model can significantly outperform MCTS when it reaches depth levels 4–5.[8] This finding demonstrates that search depth is a critical determinant of Minimax performance and, when supported by appropriate heuristics, can rival MCTS in deterministic games. The same study also points out that Connect 4's low complexity and the randomness of MCTS search policies can sometimes be disadvantageous.

Research examining the impact of heuristic functions on Minimax performance also holds significant importance in the field. Kang, Wang, and Hu's detailed heuristic analysis study demonstrates that the two main factors determining Minimax's success are search depth and the number of features used. It has been reported that different heuristic designs significantly improve performance with increasing depth, and that multiple features (single tile values, double- and triple-joint connections, spatial weights, etc.)[1] significantly improve decision quality when used together. This study highlights that dynamic move ordering, threat detection, and alpha-beta pruning are also components that significantly contribute to overall efficiency.

In Abdalaziz Sawwan's project[4], an artificial intelligence player, similar to our agent, was developed for the Connect Four game by integrating Alpha-Beta Pruning into the Minimax algorithm. The use of a unique utility function that rewards moves to the center for strategically evaluating positions is a valuable insight for our study. Furthermore, the developed model has been observed to achieve successful results against random players at low depth and against rational players at higher depth. It has been reported that the accuracy and success rates of the model's decisions during the game increase significantly with increasing depth.

Connect 4 is considered a solved game and provides a testing ground for reinforcement learning algorithms. According to Victor Allis's 1998 study, if the middle column is moved first, the player who moves first wins the game in any case. [7] On the other hand, the Q-learning and Sarsa algorithms developed in a Stanford University study were compared and tested. The results showed that the Q-learning agent achieved a maximum win rate of 73.4% against Sarsa in the scenario where the agent started as the first player, and 73.5% when Sarsa started first. [5] This is inconsistent with Allis's study and therefore demonstrates that RL algorithms deviate from optimal results and still have room for improvement.

The existing literature generally converges on three common points: the Minimax algorithm is quite strong in deterministic games when optimized with appropriate heuristics and pruning strategies; MCTS is quite stable, especially for large iterations; however, its performance depends heavily on the given time bound and inherent randomness; and move order optimizations such as move ordering, killer moves, transposition tables, and threat detection provide significant performance improvements for Minimax-based Connect 4. Therefore, these studies support the search optimizations and heuristic design that form the basis of the optimized Connect 4 agent developed in this paper.

III. Methodology

This section will explain the implemented algorithm, both mathematically and with some preliminary information. First, we will present the structure of the game and the optimization components integrated into our algorithm, followed by its basic implementation and formulation. Then, the added optimizations will be discussed and explained along with their application methods. Finally, we'll discuss our heuristic scoring formula and explain how we arrived at the decision score.

A. Preliminary

Connect4 is a deterministic, two-player, zero-sum game played on a 6×7 grid. Each state is represented as a matrix, where +1, -1, and 0 represent the AI agent, the human player, and empty cells, respectively. Legal moves occur when the disc goes to the bottom of a column with one or more spaces when you drop it.

The game's end conditions is four identical disks arranged in a row or a full board diagonal, horizontal or vertical. This structure provides the efficient use of search algorithms such as Minimax with alpha-beta pruning.

However, due to the exponential growth of the search space, the standard Minimax algorithm becomes inefficient in real-time games with high depth. To fix this issue, alpha–beta pruning, along with additional optimizations such as Move Ordering, Transposition Table, Threat Detection, Killer Moves, Evaluation Board, and Bitboard have been integrated into the algorithm to improve decision quality and computational efficiency.

B. Mathematical and Algorithmic Formalism

Let $A(s)$ denote the set of admissible actions in state s. The recursive valuation function $V(s)$ under adversarial search is defined as:

$$V(s) = \begin{cases} U(s), \text{ if } s \text{ is terminal or at depth zero} \\ \max\limits_{a \in A(s)} V(RESULT(s,a)), \text{ if the agent is maximizing} \\ \min\limits_{a \in A(s)} V(RESULT(s,a)), \text{ if the agent is minimizing} \end{cases} \quad (1)$$

where $U(s)$ denotes the utility function for terminal states. To improve search efficiency, the **alpha–beta pruning algorithm** maintains upper and lower bounds, α and β, respectively, which are dynamically updated:

$$\alpha \leftarrow \max(\alpha, V(s)), \beta \leftarrow \min(\beta, V(s)) \quad (2)$$

Pruning is applied when $\alpha \geq \beta$, signifying that further exploration of the current branch cannot influence the optimal decision. Although alpha–beta pruning does not improve the worst-case time complexity asymptotically, its performance is highly sensitive to the order in which actions are evaluated.

C. Optimization Components

1) Transposition Table:

Redundant game states often arise via distinct action sequences yielding identical board configurations. To eliminate unnecessary recomputation, a transposition table indexed by a hash function $h(s)$ is employed:

$$\boldsymbol{T[h(s)] = V(s)} \quad (3)$$

When a previously visited state is encountered, its stored evaluation is reused, effectively reducing the effective search depth and increasing consistency across subtrees.

2) Move Ordering Heuristics:

The efficacy of alpha–beta pruning is strongly influenced by the order in which child nodes are explored. To maximize

pruning potential, a composite priority scoring function $P(a)$ is utilized:

$$P(a) = Pwin \cdot Pblock \cdot Pkiller \cdot Pcenter \cdot Peval \quad (4)$$

The components are defined as follows:

- P_{win}: Immediate win detection (highest priority)
- P_{block} : Prevents immediate loss by blocking the opponent
- P_{killer}: Previously identified killer moves at the same depth
- P_{center} : Bias towards central columns to increase connectivity
- P_{eval}: Shallow heuristic score estimate

Experimentally, this prioritization strategy yields a 60–75% reduction in the number of nodes evaluated during mid-game states.

3) Threat-Aware Evaluation Function:

Unfinished games are scored using a differential heuristic. Here, $E(s)$ represents the heuristic evaluation score assigned to the unfinished game state s , where higher values indicate better positions for the AI and lower values indicate favorable positions for the opponent:

$$\boldsymbol{E(s) = EAI(s) - Eopponent(s)} \quad (5)$$

Each four-cell position is analyzed to estimate its value based on the number of AI pieces, opponent pieces, and empty slots. Patterns that indicate immediate wins or losses are given high weights. This heuristic method produces position-based predictions.

4) Killer Move Heuristic:

At each search depth d , previously effective moves that aided pruning are preserved.These "killer moves" are prioritized when revisited at the same depth, exploiting structural repetition in the game tree to speed up pruning.

5) Final Heuristic Formula and Values:

$$\text{Eval}(s) = F_{\text{pos}} + 3F_{\text{center}} + 10000F_4 + 10F_3 + 3F_2 - \left(10000F_4^{\text{opp}} + 10F_3^{\text{opp}} + 3F_2^{\text{opp}}\right) - 80F_{\text{threat3}} - 5F_{\text{threat2}} \quad (6)$$

F_{pos}: $evaluation\ board\ positional\ weights(3-13)$

F_{center}: $center\ column\ contribution(3 \times count\ difference)$

$F_4,\ F_3,\ F_2$: $AI\ four, three, and\ two\ in\ a\ row\ window\ counts$

F_4^{opp}, F_3^{opp}, F_2^{opp}: opponent four-, three-, and two-in-a-row window counts

F_{threat3}: $opponent\ 3-threat\ occurrences$

F_{threat2}: $opponent\ 2-threat\ occurrences$

Heuristic Component	Score Value
4-in-a-row (winning line)	+10,000
3-in-a-row	+10
2-in-a-row	+3
Opponent 3-in-a-row	−80
Opponent 2-in-a-row	−5
Center column bonus	+100 → +70 (gradient)
Killer move (prioritized)	+1,000,000
Immediate threat block	+5,000,000
Winning move (immediate)	+100,000,000
Evaluation board values	+3 to +13
Terminal state (win/loss)	±10,000,000 ± depth adjustment

6) Dynamic Depth

Algorithm 1: Runtime-Based Dynamic Depth Adjustment (Pseudocode)

Input:
d ← Current search depth
t_actual ← Actual thinking time (in seconds)
r ← Current round count t_target ← Target thinking time (default: 4.0)
δ ← Adjustment threshold (default: 2.5)
r_min ← Minimum rounds for depth increase (default: 4)
t_max ← Max time for decrease (default: 6.0)
d_min, d_max ← Depth bounds (e.g., 4 and 12)
Output:
d' ← Adjusted search depth
.Begin
lower ← t_target - δ // e.g., 1.5
upper ← t_target + δ // e.g., 6.5

if t_actual < lower and r ≥ r_min then
d' ← min(d + 1, d_max)
else if t_actual > upper and t_actual > t_max then
d' ← max(d - 1, d_min)
else
d' ← d
return d'
End

D. Integrated Decision Architecture

Before initiating the search, two specific conditions are evaluated:
Whether a move exists that the AI can instantly win.
Whether the opponent threatens to instantly win.
If any of these conditions are met, the relevant move is selected without further search. Otherwise, the Minimax procedure, enhanced by the optimization modules described above, is executed. This layered architecture enables decision-making at depths ranging from 8 to 12 layers, reducing computational overhead through early termination.

IV. Test and Results

Firstly to compare different variants of the alpha-beta pruning algorithm, we conducted an experimental analysis for four different configurations and examined four different configurations: Baseline without optimization, Move ordering using strategic ordering, Ordering supported by killer moves, and Full heuristics including all heuristic improvements.

Fifteen random midgame positions were examined, and searches were conducted at a depth of 6 were performed, and the average number of nodes, decision time, and pruning rates were measured.

The results show that the move ordering technique alone reduced the number of nodes by 81.2%, resulting in a 5.32x speedup. This rate reached 86.4% with killer moves and 89.8% with the full heuristics configuration. The increase in

pruning rates indicates that strategic ordering is approaching the theoretically optimal results.

These findings demonstrate that searches with a depth of 8–10 can be performed in a few seconds using domain-specific heuristic techniques, significantly improving competitive AI performance.

TABLE I. ALPHA-BETA PRUNING EFFICIENCY COMPARISON

Configuration	*Avg. Nodes*	*Avg. Time (ms)*	*Pruning Rate*	*Gain*
Baseline	8,951	1,143.2	45.5%	—
Move Ordering	1,682	393.1	63.3%	5.32× faster
Killer Moves	1,215	293.6	65.9%	7.37× faster
Full Heuristics	917	197.9	62.1%	9.76× faster

Secondly, for algorithm comparison, three algorithms were tested with six different methods: the standard alpha-beta algorithm (based on a 2D array), bitboard-optimized alpha-beta (at two different depths), and Monte Carlo Tree Search (with two different iteration numbers). Matches consisting of 10 games were conducted for each method. Win rates and average thinking times were analyzed and presented graphically.

Matchup 1: Alpha-Beta (2D, 6 Depth) clearly outperformed MCTS (10K iterations, 0.22s) with a 100% win rate and an average time of 0.87s. This demonstrates that deterministic search, supported by robust heuristics, is more effective against moderate stochastic approaches.

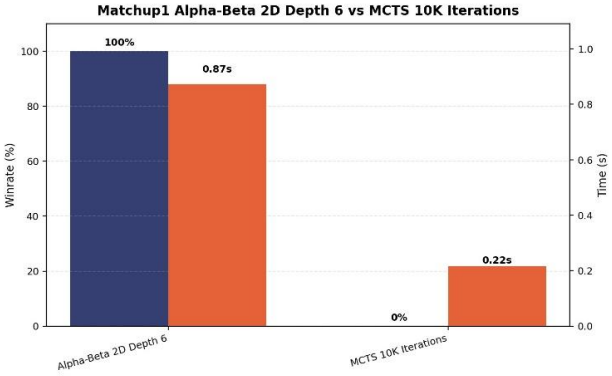

Fig. 1. Matchup 1 (Alpha-Beta(left), MCTS(right))

Matchup 2: Standard Alpha-Beta (2D, 6-depth) won all games against Bitboard. Bitboard's overly aggressive cache usage caused it to repeat the same incorrect moves, resulting in a very low win rate.

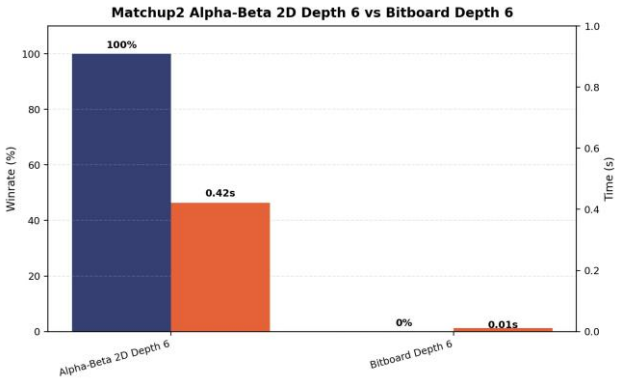

Fig. 2. Matchup 2 (Alpha-Beta(left), BitBoard(Right))

Matchup 3: Bitboard (6-depth) vs. MCTS: In a 10K iteration match, Bitboard beat MCTS in all games with a 13x speed advantage. Bitboard's high-speed computational capabilities outperformed MCTS's shallow tree.

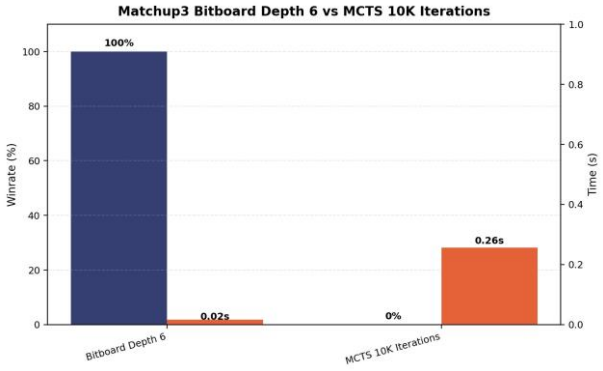

Fig. 3. Matchup 3 (BitBoard(left), MCTS(Right))

Matchup 4: Despite 50K iterations, MCTS lost all games against Alpha-Beta (2D, 6-depth). This demonstrates that despite the high iteration count, MCTS cannot match the gaming power of well-structured deterministic search.

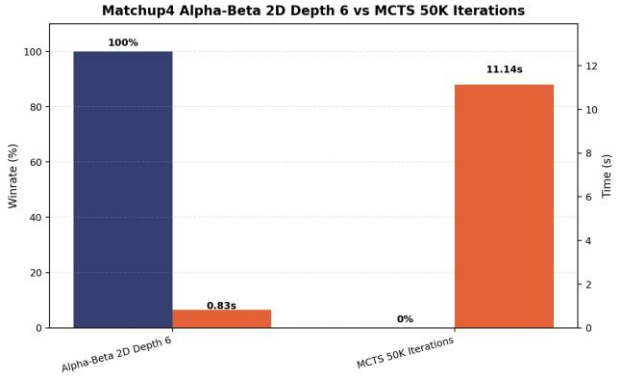

Fig. 4. Matchup 4 (Alpha-Beta(left), MCTS(Right))

Matchup 5: Bitboard (10 depth) achieved a 90% win rate against MCTS (50K iterations). MCTS won only one game, but its performance was observed to be inefficient with a time of 8.04s. While higher iterations could provide very rare tactical opportunities, these opportunities were found to be less effective in overall success.

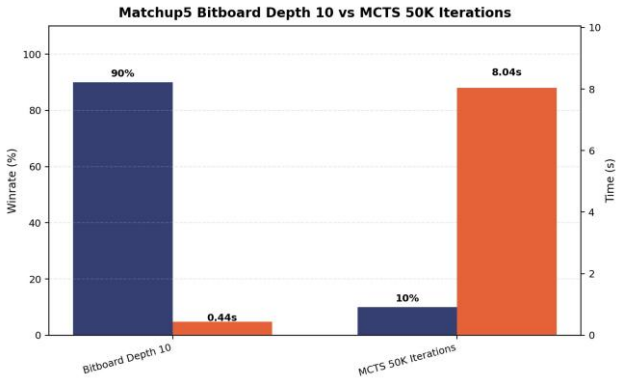

Fig. 5. Matchup 5 (BitBoard(left), MCTS(Right))

Matchup 6: Alpha-Beta (2D, 6 Depth) won all games against the deeper and faster Bitboard (10 Depth). Bitboard's excessive cache optimization led to aggressive play, which led to a series of losses.

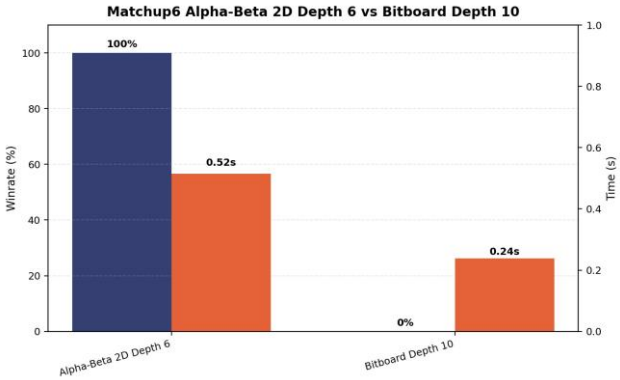

Fig. 6. Matchup 6 (Alpha-Beta(left), BitBoard(Right))

The experimental results led us to three main conclusions. First of all, the alpha-beta pruning algorithm repeatedly outperformed MCTS. Deterministic search, supported by heuristics that prioritize threat detection and central dominance, surpasses the stochastic approach in terms of strategic depth.

Secondly, Bitboard's aggressive cache mechanism's desire for quick wins and swift defenses prevented it from winning games. However, in pruning efficiency tests, the same framework delivered a nearly 10x speedup.

Thirdly and finally, high speed alone wasn't enough. While Bitboard could search deeper and faster , the simpler and slower alpha-beta (depth-6) achieved greater playing power and game win rate. This difference stems from the well-balanced and robust evaluation functions.

These results demonstrate that focusing on more meaningful optimizations that can be combined, such as move sequencing, killer moves, and threat analysis, rather than low-level optimizations, has a more positive impact on the overall outcome.

V. Conclusion and Future Work

This project focuses on heuristic approaches and algorithms to achieve competitive AI performance in the Connect Four game. A particular focus is on the alpha-beta pruning technique. Optimization methods such as move ordering and dynamic depth have been integrated to improve the search process. These techniques have yielded beneficial performance improvements.

The results demonstrate that AI can outperform humans even at lower depths thanks to such optimizations. While the algorithms used are theoretically available in the literature, the unique adaptations implemented in this study demonstrate that improvements can be made to classical methods with different approaches. In heuristic systems, the balance between general approaches and situation-specific fine-tuning is delicate. However, in games with clear rules and structural boundaries, such as Connect Four, this balance often tends to shift towards improving performance.

Future work includes integrating an "opening book" system that can pre-analyze opening positions and offer move suggestions. Furthermore, porting performance-critical components to a lower-level environment like C++ could improve the agent, particularly in terms of speed.

This project demonstrates how deep and meaningful a board game can be, how it can encourage different thinking, and how suitable it can be for the testing environment of artificial intelligence.

The full implementation of the proposed agent are available on GitHub:
https://github.com/oguzhansarigol/connect4-ai-agent

Acknowledgment

This study has been done in the frame of Eskişehir Osmangazi University 152117107 Introduction to Artificial Intelligence course by the supervision of Asst. Prof. Savaş Okyay.

References

[1] X. Kang, Y. Wang, and Y. Hu, “Research on Different Heuristics for Minimax Algorithm Insight from Connect-4 Game,” Journal of Intelligent Learning Systems and Applications, vol. 11, pp. 15–31, 2019.

[2] H. Taylor and L. Stella, “An Evolutionary Framework for Connect-4 as Test-Bed for Comparison of Advanced Minimax, Q-Learning and MCTS,” arXiv preprint arXiv:2405.16595, 2024. [Online]. Available: https://arxiv.org/pdf/2405.16595

[3] K. Sheoran, G. Dhand, M. Dabas, N. Dahiya, and P. Pushparaj, “Solving Connect 4 Using Optimized Minimax and Monte Carlo Tree Search,” Advances and Applications in Mathematical Sciences, vol. 21, no. 6, pp. 3303–3313, Apr. 2022.

[4] A. Sawwan, Artificial Intelligence-Based Connect 4 Player Using Python, AI Course Project, supervised by Dr. Pei Wang, Temple University, 2021.

[5] E. Alderton, E. Wopat, and J. Koffman, Reinforcement Learning for Connect Four, Stanford University, Final Report for CS 238, 2019. [Online]. Available: https://web.stanford.edu/class/aa228/reports/2019/final106.pdf

[6] J. Tromp, Connect Four Solver, M.Sc. thesis, University of Alberta, 1995. [Online]. Available: https://tromp.github.io/c4/connect4_thesis.pdf

[7] V. Allis, *A Knowledge-Based Approach of Connect-Four: The Game is Solved — White Wins*, M.Sc. thesis, Dept. Mathematics and Computer Science, Vrije Universiteit Amsterdam, The Netherlands, Oct. 1988.

[8] K. Sheoran, G. Dhand, M. Dabaszs, N. Dahiya and P. Pushparaj, “Solving Connect 4 using optimized Minimax and Monte Carlo Tree Search,” in 2023 International Conference on Intelligent Systems and Computation(ISC), IEEE, 2023